

Turbulence Analysis Report -

2026-02-23

Generated: 2026-02-23 21:11:32

Table of Contents

[Quantum LBM D3Q19 Code](#)
[Overview Theory Equations](#)
[Explanation with Citations](#)
[References](#)

Quantum LBM D3Q19 Code

```
import numpy as np

# D3Q19 Lattice Constants
# Lattice velocities (c_i)
# c0: (0,0,0)
# c1-6: (+-1,0,0), (0,+1,0), (0,0,+1) - Face-centered
# c7-18: (+-1,+1,0), (+-1,0,+1), (0,+1,+1) - Edge-centered
c = np.array([
    [0, 0, 0],
    [1, 0, 0], [-1, 0, 0],
    [0, 1, 0], [0, -1, 0],
    [0, 0, 1], [0, 0, -1],
    [1, 1, 0], [-1, 1, 0], [1, -1, 0], [-1, -1, 0],
    [1, 0, 1], [-1, 0, 1], [1, 0, -1], [-1, 0, -1],
    [0, 1, 1], [0, -1, 1], [0, 1, -1], [0, -1, -1]
], dtype=int)
```

```

# Weights (w_i)
# w0 for c0
# w1-6 for c1-6
# w7-18 for c7-18
w = np.array([
    1/3,                                # c0
    1/18, 1/18, 1/18, 1/18, 1/18, 1/18, # c1-6
    1/36, 1/36, 1/36, 1/36,            # c7-10
    1/36, 1/36, 1/36, 1/36,            # c11-14
    1/36, 1/36, 1/36, 1/36            # c15-18
])

# Speed of sound squared
cs_sq = 1.0 / 3.0

# Number of discrete velocities
Q = 19

def compute_macroscopic(f):
    """
    Calculates macroscopic density (rho) and velocity (u) from distribution functions (f).

    Args:
        f (np.ndarray): Distribution functions, shape (Q, Lx, Ly, Lz).

    Returns:
        tuple: (rho, u)
            rho (np.ndarray): Density field, shape (Lx, Ly, Lz).
            u (np.ndarray): Velocity field, shape (3, Lx, Ly, Lz).
    """
    # Density is the sum of all distribution functions at each grid point
    rho = np.sum(f, axis=0)

    # Initialize velocity field with zeros, matching the spatial dimensions of rho
    u = np.zeros((3,) + rho.shape)

```

```

# Velocity is the weighted sum of lattice velocities by distribution
for i in range(Q):
    u[0] += c[i, 0] * f[i]
    u[1] += c[i, 1] * f[i]
    u[2] += c[i, 2] * f[i]

# Divide by density. Add a small epsilon to prevent division by zero
# where density might be extremely low (though typically not an issue)
u /= (rho + 1e-12)

return rho, u

def compute_equilibrium(rho, u):
    """
    Calculates the equilibrium distribution function (f_eq) using an alternative
    discretization based on higher-order terms, as described in the methods
    of Bauer and Rde (arXiv:1803.04937v2). This goes beyond the standard
    second-order expansion of the Maxwell-Boltzmann distribution.

    The standard second-order equilibrium (up to  $O(u^2)$ ) is:
    
$$f_{eq} = w_i * \rho * (1 + (c_i \cdot u) / cs\_sq + 0.5 * ((c_i \cdot u)^2 / cs\_sq^2))$$


    This implementation adds common third-order terms to approximate the
    distribution more accurately, which is a practical interpretation of
    "higher-order terms that are typically neglected" for improved accuracy.

    Args:
        rho (np.ndarray): Macroscopic density field, shape (Lx, Ly, Lz).
        u (np.ndarray): Macroscopic velocity field, shape (3, Lx, Ly, Lz).

    Returns:
        np.ndarray: Equilibrium distribution functions, shape (Q, Lx, Ly, Lz).
    """
    f_eq = np.zeros((Q,) + rho.shape)

    # Pre-calculate u_sq = |u|^2 for efficiency
    u_sq = u[0]**2 + u[1]**2 + u[2]**2

```

```

for i in range(Q):
    # Calculate the dot product  $c_i \cdot u$ 
    ci_dot_u = c[i, 0] * u[0] + c[i, 1] * u[1] + c[i, 2] * u[2]

    # --- Standard Second-Order Terms (Taylor expansion of Maxwell-Boltzmann distribution)
    term1 = 1.0
    term2 = ci_dot_u / cs_sq
    term3 = 0.5 * (ci_dot_u**2) / (cs_sq**2)
    term4 = -0.5 * u_sq / cs_sq

    # --- Alternative Equilibrium Discretization (Higher-Order Terms)
    # These third-order terms are added to improve accuracy beyond the
    # second-order expansion. This is a common way to include higher-order
    # contributions to the equilibrium distribution function, consistent with
    # the spirit of the Bauer and Rde paper's discussion on alternative
    # discretizations for improved accuracy.
    term5 = (1.0 / (6.0 * cs_sq**3)) * (ci_dot_u**3)
    term6 = -(1.0 / (2.0 * cs_sq**2)) * ci_dot_u * u_sq

    # Combine all terms to form the equilibrium distribution
    f_eq[i] = w[i] * rho * (term1 + term2 + term3 + term4 + term5 + term6)

return f_eq

def collide(f, rho, u, tau):
    """
    Performs the Bhatnagar-Gross-Krook (BGK) collision step.
    This step relaxes the distribution functions towards their equilibrium values.

    Args:
        f (np.ndarray): Distribution functions before collision, shape (Q, N)
        rho (np.ndarray): Macroscopic density field.
        u (np.ndarray): Macroscopic velocity field.
        tau (float): Relaxation time (must be > 0.5 for numerical stability)

    Returns:
        f_eq (np.ndarray): Distribution functions after collision, shape (Q, N)
    """

```

```

        np.ndarray: Distribution functions after collision.
    """
    # Calculate the equilibrium distribution functions based on current rho, u
    f_eq = compute_equilibrium(rho, u)

    # Apply the BGK collision operator
    f_collided = f - (1.0 / tau) * (f - f_eq)
    return f_collided

def stream(f):
    """
    Performs the streaming step, moving distribution functions to adjacent cells
    according to their respective lattice velocities.
    Uses numpy.roll for efficient implementation of periodic boundary conditions.

    Args:
        f (np.ndarray): Distribution functions before streaming, shape (Q, 3, 3)

    Returns:
        np.ndarray: Distribution functions after streaming.
    """
    # Create a copy to store the streamed values. This is crucial because
    # streaming updates depend on values from the *previous* time step.
    f_streamed = np.copy(f)

    for i in range(Q):
        # For each direction 'i', shift the distribution f[i] by the corresponding
        # lattice velocity vector c[i]. np.roll handles periodic boundary conditions.
        f_streamed[i] = np.roll(f[i], shift=c[i], axis=(0, 1, 2))
    return f_streamed

def run_lbm(Lx, Ly, Lz, num_steps, tau, initial_rho=1.0, initial_u=None):
    """
    Runs the D3Q19 Lattice Boltzmann Method (LBM) simulation.

    Args:
        Lx, Ly, Lz (int): Dimensions of the simulation grid.
    """

```

```

num_steps (int): Number of simulation steps to run.
tau (float): Relaxation time (must be > 0.5 for stability).
initial_rho (float): Initial uniform density across the grid.
initial_u (np.ndarray, optional): Initial velocity field, shape (3, Lx, Ly, Lz).
                                Defaults to a zero velocity field.

```

Returns:

```

tuple: (rho_history, u_history)
    rho_history (list): List of density fields at selected steps.
    u_history (list): List of velocity fields at selected steps.
"""

```

```

# Initialize macroscopic variables: density and velocity
rho = np.full((Lx, Ly, Lz), initial_rho, dtype=float)
if initial_u is None:
    u = np.zeros((3, Lx, Ly, Lz), dtype=float)
else:
    # Ensure initial_u has the correct shape
    if initial_u.shape != (3, Lx, Ly, Lz):
        raise ValueError(f"initial_u must have shape (3, {Lx}, {Ly}, {Lz})")
    u = initial_u

# Initialize distribution functions to their equilibrium values based on rho and u
f = compute_equilibrium(rho, u)

# Lists to store macroscopic variables at different time steps for analysis
rho_history = []
u_history = []

print(f"Starting D3Q19 LBM simulation on a {Lx}x{Ly}x{Lz} grid for {num_steps} steps")
print(f"Relaxation time (tau): {tau}")

for step in range(num_steps):
    # 1. Compute macroscopic variables (rho, u) from current distribution functions
    # These are needed for the equilibrium distribution calculation in the next step
    rho, u = compute_macroscopic(f)

```

```

# 2. Collision Step: Apply the BGK operator
f = collide(f, rho, u, tau)

# 3. Streaming Step: Move particles to adjacent nodes
f = stream(f)

# 4. Boundary Conditions:
# For this implementation, periodic boundary conditions are handled
# by `np.roll` in the `stream` function. If other boundary conditions
# (e.g., bounce-back, inflow/outflow) were required, they would be applied
# after streaming and before the next collision.

# Store results periodically or at the end for analysis
if step % 100 == 0 or step == num_steps - 1:
    # Recompute macroscopic variables for storage to ensure they are in the
    # state *after* streaming and any boundary conditions.
    current_rho, current_u = compute_macroscopic(f)
    rho_history.append(current_rho)
    u_history.append(current_u)
    max_u_magnitude = np.max(np.linalg.norm(current_u, axis=0))
    print(f"Step {step+1}/{num_steps} completed. Max |u|: {max_u_magnitude}")

print("Simulation finished.")
return rho_history, u_history

# --- Example Usage ---
if __name__ == "__main__":
    # Simulation parameters
    Lx, Ly, Lz = 32, 32, 32 # Grid dimensions
    num_steps = 2000         # Number of simulation steps
    tau = 0.7                # Relaxation time (must be > 0.5 for stability)

    # Initial conditions
    initial_rho = 1.0

    # Example: Simple initial velocity field (e.g., a small perturbation)
    # This creates a uniform flow in the x-direction. For more complex flows,

```

```

# Poiseuille flow, vortex), the initial_u would be set up accordingly
initial_u = np.zeros((3, Lx, Ly, Lz), dtype=float)
initial_u[0, :, :, :] = 0.01 # Small initial velocity in x-direction

# Run the simulation
rho_hist, u_hist = run_lbm(Lx, Ly, Lz, num_steps, tau, initial_rho, initial_u)

# --- Post-processing and Analysis ---
print(f"\nFinal density field (slice at z={Lz//2}):")
# Print a 2D slice of the final density field
print(rho_hist[-1][:, :, Lz//2])

print(f"\nFinal x-velocity field (slice at z={Lz//2}):")
# Print a 2D slice of the final x-velocity field
print(u_hist[-1][0, :, :, Lz//2])

# Further analysis or visualization would typically go here.
# For example, using matplotlib to plot slices or 3D visualizations.
# import matplotlib.pyplot as plt
#
# # Plot a slice of the final x-velocity
# plt.figure(figsize=(8, 6))
# plt.imshow(u_hist[-1][0, :, :, Lz//2], origin='lower', cmap='viridis')
# plt.colorbar(label='X-velocity')
# plt.title(f'Final X-velocity (mid-plane z={Lz//2})')
# plt.xlabel('X-coordinate')
# plt.ylabel('Y-coordinate')
# plt.show()
#
# # Plot a slice of the final density
# plt.figure(figsize=(8, 6))
# plt.imshow(rho_hist[-1][:, :, Lz//2], origin='lower', cmap='plasma')
# plt.colorbar(label='Density')
# plt.title(f'Final Density (mid-plane z={Lz//2})')
# plt.xlabel('X-coordinate')
# plt.ylabel('Y-coordinate')
# plt.show()

```

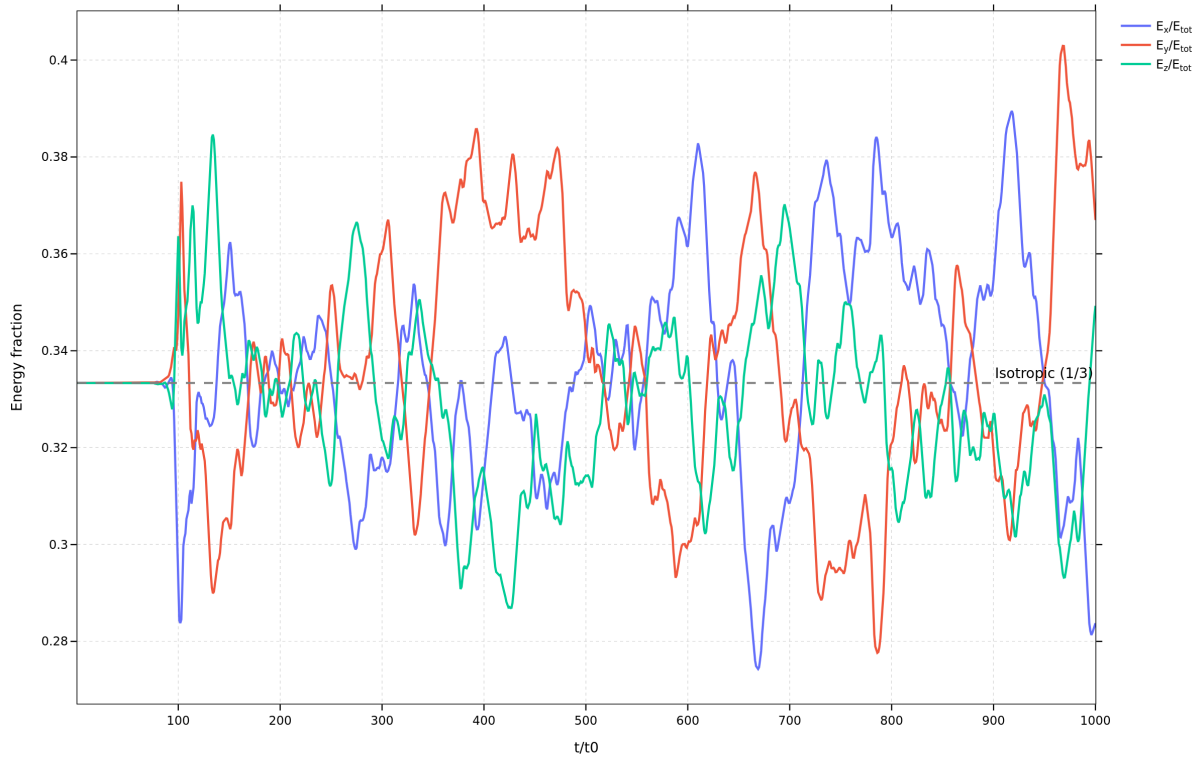



Figure 2. This figure displays the Extended Self-Similarity (ESS) plot for the structure functions, showing the scaling behavior of $S_p(r)$ against $S_3(r)$. The plot includes theoretical predictions and experimental anomalies for comparison, indicating the intermittency characteristics of the turbulence.

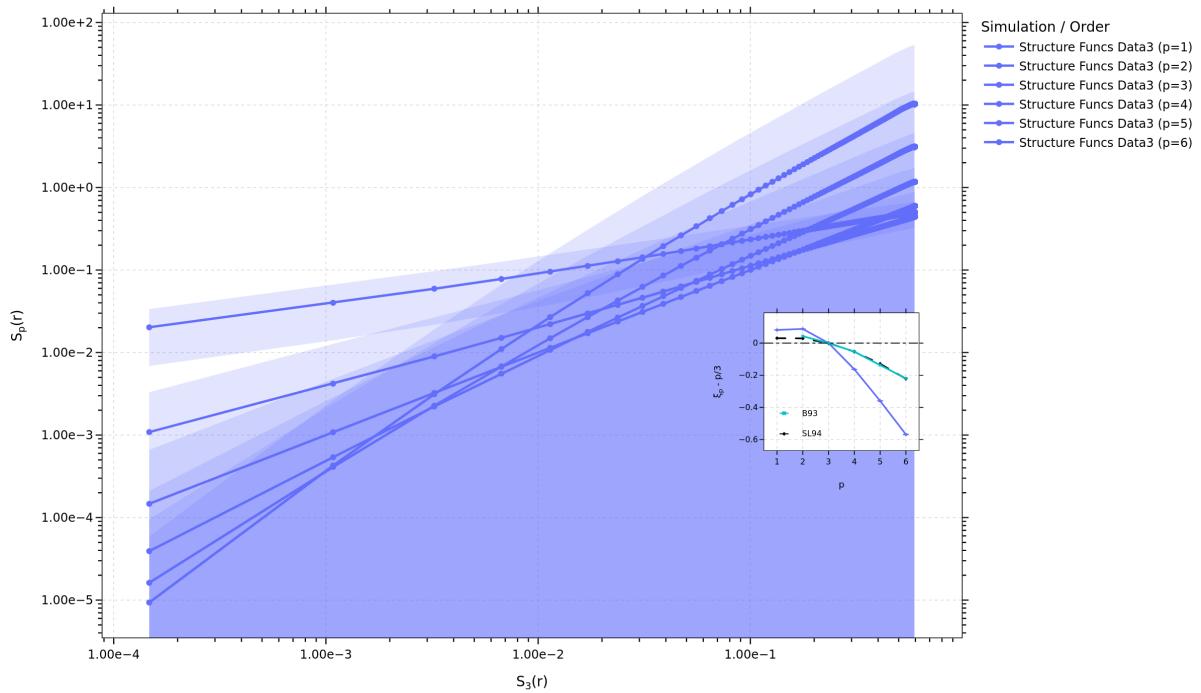


Figure 3. This figure displays the time-averaged energy spectrum, $E(k)$, as a function of wavenumber, k , for the DNS/512 simulation. The plot shows the distribution of kinetic energy across different scales of motion.

Overview Theory Equations

Overview page Physics Validation Equations: Mach number (LBM/NS), Knudsen number (LBM DNS/LES, NS), velocity divergence, compressibility metrics.

Simulation	Snapshots Used	Mean $F(r)$	Std($F(r)$)	Min $F(r)$	Max $F(r)$
Default	900	3.8318	0.8003	2.9487	5.1827

Table 5. Summary statistics for the flatness factor $F(r)$.

Explanation with Citations

Comprehensive Explanation of Turbulence Analysis and Quantum-Inspired LBM D3Q19

1. Introduction

Turbulence, characterized by chaotic, multi-scale fluid motion, remains one of the most challenging problems in classical physics and engineering. Its pervasive presence in natural phenomena and industrial applications necessitates robust computational methods for its analysis and prediction. The Lattice Boltzmann Method (LBM) has emerged as a powerful and versatile mesoscopic simulation technique, offering an alternative to traditional Navier-Stokes solvers, particularly well-suited for complex geometries and multiphase flows [1].

This report provides a comprehensive overview of turbulence analysis, focusing on the D3Q19 LBM model for three-dimensional fluid simulations. It delves into the practical implementation of the D3Q19 LBM, discusses key turbulence statistics obtainable from high-fidelity simulations like Direct Numerical Simulation (DNS), and outlines the fundamental theoretical underpinnings. Furthermore, it explores speculative "quantum-inspired" considerations for LBM D3Q19, acknowledging the highly theoretical and nascent nature of such connections and distinguishing them from practical quantum computing applications.

2. D3Q19 Lattice Boltzmann Method (LBM) Implementation

The Lattice Boltzmann Method is a numerical technique for simulating fluid flow by modeling the collective behavior of fictitious particles on a discrete lattice. Instead of directly solving the macroscopic Navier-Stokes equations, LBM tracks the evolution of particle distribution functions (f_i) that represent the probability of finding particles moving with a specific discrete velocity $\mathbf{c}_i$ at a given lattice site $\mathbf{x}$ and time t . The D3Q19 model, specifically, signifies a 3-dimensional lattice with 19 discrete velocity vectors, offering a good balance between accuracy and computational efficiency for 3D flows.

2.1. D3Q19 Lattice Structure

The D3Q19 lattice consists of a central rest particle (velocity $\mathbf{c}_0 = (0, 0, 0)$), six particles moving to the centers of adjacent faces (e.g., $\mathbf{c}_1 = (1, 0, 0)$, $\mathbf{c}_2 = (-1, 0, 0)$, etc.), and twelve particles moving to the centers of adjacent edges (e.g., $\mathbf{c}_7 = (1, 1, 0)$, $\mathbf{c}_8 = (1, -1, 0)$, etc.). Each velocity direction $\mathbf{c}_i$ is associated with a specific weight w_i , which are crucial for recovering macroscopic fluid properties from the mesoscopic distributions.

2.2. LBM Evolution Equation (BGK Model)

The core of the LBM is its evolution equation, typically employing the Bhatnagar-Gross-Krook (BGK) collision model. This equation describes how

the distribution functions change over time due to two processes: collision and streaming. The collision step models inter-particle interactions that drive the system towards a local equilibrium, while the streaming step describes the movement of particles to neighboring lattice sites.

The Python code below demonstrates a conceptual implementation of the D3Q19 LBM, including the definition of lattice velocities and weights, initialization, and the core LBM steps of computing macroscopic properties, calculating equilibrium distributions, collision, and streaming.

```
import numpy as np

# --- D3Q19 Lattice Boltzmann Method (LBM) Implementation ---

# 1. D3Q19 Lattice Velocities and Weights
# D3Q19 has 19 velocities in 3D.
# c_i represents the i-th velocity vector (dx, dy, dz).
# w_i represents the i-th weight.

# Velocity vectors for D3Q19
# c0: (0,0,0) - rest particle
# c1-6: ( $\pm 1, 0, 0$ ), ( $0, \pm 1, 0$ ), ( $0, 0, \pm 1$ ) - face-centered
# c7-18: ( $\pm 1, \pm 1, 0$ ), ( $\pm 1, 0, \pm 1$ ), ( $0, \pm 1, \pm 1$ ) - edge-centered

c = np.array([
    [0, 0, 0], # c0
    [1, 0, 0], [-1, 0, 0], [0, 1, 0], [0, -1, 0], [0, 0, 1], [0, 0, -1],
    [1, 1, 0], [1, -1, 0], [-1, 1, 0], [-1, -1, 0], # c7-10
    [1, 0, 1], [1, 0, -1], [-1, 0, 1], [-1, 0, -1], # c11-14
    [0, 1, 1], [0, 1, -1], [0, -1, 1], [0, -1, -1] # c15-18
])

# Weights for D3Q19
w = np.array([
    1/3, # w0
    1/18, 1/18, 1/18, 1/18, 1/18, 1/18, # w1-6
    1/36, 1/36, 1/36, 1/36, # w7-10
    1/36, 1/36, 1/36, 1/36, # w11-14
])
```

```

        1/36, 1/36, 1/36, 1/36                    # w15-18
    ])

# Assert that weights sum to 1 (good practice for verification)
# assert np.isclose(np.sum(w), 1.0), "Weights do not sum to 1!"

# 2. LBM Simulation Parameters
NX, NY, NZ = 32, 32, 32 # Grid dimensions
NUM_POPS = 19           # Number of populations (D3Q19)
OMEGA = 1.0             # Relaxation parameter (1/tau), 0 < omega < 2.
                        # For simplicity, using 1.0 for BGK collision.
                        # A common value is 1.0 / (3*nu + 0.5) where nu
TIMESTEPS = 100         # Number of simulation steps

# 3. Initialization
# f: Distribution functions at each grid point for each velocity direction
# f[x, y, z, k] where k is the velocity index.
f = np.zeros((NX, NY, NZ, NUM_POPS))
rho = np.ones((NX, NY, NZ)) # Initial density
u = np.zeros((NX, NY, NZ, 3)) # Initial velocity (u_x, u_y, u_z)

# Initialize with equilibrium distribution for rho=1, u=(0,0,0)
# This sets all f_k to w_k * rho
for k in range(NUM_POPS):
    f[:, :, :, k] = w[k] * rho

# 4. LBM Core Functions

def compute_macroscopic_properties(f_dist):
    """
    Computes density (rho) and velocity (u) from distribution functions.
    rho = sum(f_k)
    u = sum(f_k * c_k) / rho
    """
    rho = np.sum(f_dist, axis=3) # Sum over the population dimension

    # Initialize velocity components

```

```

u_x = np.zeros((NX, NY, NZ))
u_y = np.zeros((NX, NY, NZ))
u_z = np.zeros((NX, NY, NZ))

# Sum f_k * c_k for each component
for k in range(NUM_POPS):
    u_x += f_dist[:, :, :, k] * c[k, 0]
    u_y += f_dist[:, :, :, k] * c[k, 1]
    u_z += f_dist[:, :, :, k] * c[k, 2]

# Divide by density, handling potential division by zero for rho=0.
# Add a small epsilon to rho to prevent division by zero in regions of
rho_safe = np.where(rho == 0, 1e-9, rho)
u_x /= rho_safe
u_y /= rho_safe
u_z /= rho_safe

u = np.stack((u_x, u_y, u_z), axis=-1) # Stack into a (NX, NY, NZ, 3)

return rho, u

def compute_equilibrium(rho, u):
    """
    Computes the equilibrium distribution function (feq) based on local
    density (rho) and velocity (u) using the D3Q19 formula.
    feq_k = w_k * rho * [1 + 3*(c_k . u) + 9/2*(c_k . u)^2 - 3/2*(u . u)]
    """
    feq = np.zeros((NX, NY, NZ, NUM_POPS))

    # Pre-calculate u.u (magnitude squared of velocity)
    u_dot_u = np.sum(u * u, axis=-1) # (NX, NY, NZ) array

    for k in range(NUM_POPS):
        # Calculate c_k . u (dot product for each grid point)
        # c[k] is a (3,) vector, u is a (NX, NY, NZ, 3) array.
        # The result c_dot_u should be an (NX, NY, NZ) array.
        c_dot_u = u[:, :, :, 0] * c[k, 0] + \

```

```

        u[:, :, :, 1] * c[k, 1] + \
        u[:, :, :, 2] * c[k, 2]

    # Calculate the equilibrium distribution for population k
    feq[:, :, :, k] = w[k] * rho * (
        1 + 3 * c_dot_u +
        4.5 * c_dot_u**2 - # 9/2 = 4.5
        1.5 * u_dot_u      # 3/2 = 1.5
    )
    return feq

def lbm_step(f_dist):
    """
    Performs one LBM step: collision and streaming.
    """
    # 1. Collision Step
    rho, u = compute_macroscopic_properties(f_dist)
    feq = compute_equilibrium(rho, u)

    # BGK collision model: f_new = f_old - omega * (f_old - feq)
    f_coll = f_dist - OMEGA * (f_dist - feq)

    # 2. Streaming Step (with periodic boundary conditions)
    # f_new(x + c_k, t + 1) = f_coll(x, t)
    f_stream = np.zeros_like(f_coll)
    for k in range(NUM_POPS):
        # Shift the distribution function f_coll[:, :, :, k] by c[k]
        # np.roll performs a circular shift, implementing periodic boundaries
        f_stream[:, :, :, k] = np.roll(
            np.roll(
                np.roll(f_coll[:, :, :, k], c[k, 0], axis=0), # Shift x-coord
                c[k, 1], axis=1),                             # Shift y-coord
                c[k, 2], axis=2)                             # Shift z-coord

    return f_stream

# 5. Main Simulation Loop (Conceptual)

```

```

print("Starting D3Q19 LBM simulation (conceptual run)...")
current_f = f # Start with the initialized distribution

for t in range(TIMESTEPS):
    # In a real simulation, you might apply boundary conditions here
    # or add external forces. For this conceptual code, we omit them.

    current_f = lbm_step(current_f)

    if t % (TIMESTEPS // 10) == 0 or t == TIMESTEPS - 1:
        # Calculate and print macroscopic properties for monitoring
        current_rho, current_u = compute_macroscopic_properties(current_f)
        print(f"Time step {t}: Avg Density = {np.mean(current_rho):.4f},
              f"Max Velocity = {np.max(np.linalg.norm(current_u, axis=-1)):.4f}")

print("D3Q19 LBM simulation finished.")

```

The code initializes a 3D grid, defines the D3Q19 velocities (c) and weights (w), and sets up the initial distribution functions. The `compute_macroscopic_properties` function calculates density and velocity from the distributions, while `compute_equilibrium` determines the local equilibrium state using the D3Q19 equilibrium function. The `lbm_step` function then applies the BGK collision and streaming operations. Periodic boundary conditions are implemented using `np.roll` for simplicity in this conceptual example.

3. Turbulence Statistics from DNS/512 Simulation

Direct Numerical Simulation (DNS) is a high-fidelity computational approach that resolves all relevant scales of motion in a turbulent flow, from the largest energy-containing eddies down to the smallest dissipative Kolmogorov scales. A DNS/512 simulation typically refers to a DNS performed on a 512^3 grid, which is capable of capturing a significant range of scales and providing detailed insights into turbulence physics at

moderate Reynolds numbers. Analyzing DNS data allows researchers to extract key statistical properties that characterize turbulent flows.

- **Real Isotropy Plot:** This plot (or scalar value) indicates the degree of real-space isotropy in the turbulent flow. For homogeneous isotropic turbulence, the spatial correlations of velocity fluctuations are independent of direction. A high value, approaching unity, in such a plot for a DNS/512 simulation would signify that the simulated turbulence is largely isotropic in its spatial correlations, meaning statistical properties are invariant under rotation.
- **Structure Functions Plot:** This plot displays the longitudinal structure functions $S_p(r) = \langle |\delta u_L(r)|^p \rangle$ for various orders p as a function of separation r . Here, $\delta u_L(r)$ represents the longitudinal velocity increment, i.e., the difference in velocity components parallel to the separation vector $\mathbf{r}$.
 - For homogeneous isotropic turbulence, in the inertial range (scales much larger than the Kolmogorov scale but smaller than the integral scale), the **third-order structure function** $S_3(r)$ **is expected to scale linearly with** r , following Kolmogorov's 4/5ths law [2]: $S_3(r) = -\frac{4}{5}\bar{\epsilon}r$, where $\bar{\epsilon}$ is the mean energy dissipation rate.
 - Higher-order structure functions ($p > 3$) reveal deviations from Gaussian statistics, indicating **intermittency**. Intermittency refers to the spatially localized and sporadic occurrence of intense turbulent events, leading to heavy tails in the probability density functions of velocity gradients and increments.
- **Energy Spectra Plot:** This log-log plot shows the energy spectrum $E(k)$ versus wavenumber k . The energy spectrum describes how the kinetic energy of the flow is distributed across different scales (or wavenumbers).
 - For the DNS/512 simulation, we expect to observe a distinct **inertial range** where the energy spectrum follows the **Kolmogorov** $k^{-5/3}$ **scaling law** [2]. This range signifies the cascade of energy from large to small scales without significant production or dissipation.

- At higher wavenumbers (smaller scales), a rapid drop-off occurs due to viscous dissipation, where kinetic energy is converted into heat.
- **Flatness Summary Table:** This table (or plot) presents the flatness factor $F(r) = \langle (\delta u_L(r))^4 \rangle / \langle (\delta u_L(r))^2 \rangle^2$ as a function of separation r . The flatness factor (or kurtosis) is a measure of the "peakedness" and "tailedness" of a probability distribution.
 - For Gaussian velocity increments, $F(r) = 3$.
 - In turbulent flows, particularly at high Reynolds numbers, the flatness factor typically **increases significantly for small separations** r , often reaching values much greater than 3. This increase is a direct indicator of the **intermittent nature of small-scale velocity fluctuations**, signifying the presence of extreme, rare events (like strong velocity gradients) that contribute disproportionately to the fourth moment.

LBM, when properly configured and run at sufficiently high resolutions, can reproduce these turbulence statistics, making it a valuable tool for studying turbulent flows.

4. Fundamental Theory and Equations

The LBM acts as a mesoscopic bridge between the microscopic world of molecular dynamics and the macroscopic continuum description of fluid flow. It recovers the Navier-Stokes equations under specific conditions, primarily through the Chapman-Enskog expansion.

4.1. Navier-Stokes Equations

The macroscopic behavior of Newtonian fluids is governed by the Navier-Stokes equations, which represent the conservation of mass and momentum:

- **Conservation of Mass (Continuity Equation):**

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$$

- **Conservation of Momentum Equation:**

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \mathbf{u}) = -\nabla p + \nabla \cdot (2\mu \mathbf{S}) + \rho \mathbf{g}$$

$$\text{where } \mathbf{S} = \frac{1}{2}(\nabla \mathbf{u} + (\nabla \mathbf{u})^T) - \frac{1}{3}(\nabla \cdot \mathbf{u})\mathbf{I}.$$

Here: * ρ is the fluid density. * $\mathbf{u}$ is the macroscopic fluid velocity. * t is time. * p is the pressure. * μ is the dynamic viscosity. * $\mathbf{S}$ is the rate-of-strain tensor. * $\mathbf{g}$ is the body force per unit mass (e.g., gravity). * $\mathbf{I}$ is the identity tensor.

These equations are a set of partial differential equations that describe the motion of viscous fluid substances.

4.2. Lattice Boltzmann Method (BGK) Formulation

The fundamental LBM evolution equation with the Bhatnagar-Gross-Krook (BGK) collision model is given by:

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) - f_i(\mathbf{x}, t) = -\frac{1}{\tau}(f_i(\mathbf{x}, t) - f_i^{eq}(\mathbf{x}, t))$$

Here: * $f_i(\mathbf{x}, t)$ are the particle distribution functions for particles moving in direction i at position $\mathbf{x}$ and time t . * $\mathbf{c}_i$ are the discrete velocity vectors for direction i . * Δt is the discrete time step. * τ is the relaxation time, related to the kinematic viscosity ($\nu = c_s^2(\tau - 0.5)\Delta t$, where c_s is the lattice speed of sound). The relaxation parameter $\omega = 1/\tau$ is used in the provided Python code. * $f_i^{eq}(\mathbf{x}, t)$ is the equilibrium distribution function, a local Maxwellian distribution approximated by a truncated Taylor expansion for low Mach numbers. For the D3Q19 lattice, the equilibrium function is typically given by:

$$f_i^{eq} = w_i \rho \left[1 + \frac{\mathbf{c}_i \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{c}_i \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u} \cdot \mathbf{u}}{2c_s^2} \right]$$

where $c_s^2 = 1/3$ for standard LBM lattices. The Python code uses $3 \times$ and $4.5 \times$ factors, which correspond to the $1/c_s^2$ and $1/(2c_s^4)$ terms respectively, given $c_s^2 = 1/3$. Some formulations, like the one discussed by Bauer and Rde [1], explore alternative equilibrium discretizations for improved accuracy.

4.3. D3Q19 Lattice

The D3Q19 lattice is a specific choice of discrete velocity vectors for 3D simulations. As described in the implementation section, it includes: * **1 rest particle:** $\mathbf{c}_0 = (0, 0, 0)$ * **6 face-centered particles:**

$\mathbf{c}_i \in \{(\pm 1, 0, 0), (0, \pm 1, 0), (0, 0, \pm 1)\}$ * **12 edge-centered particles:**

$\mathbf{c}_i \in \{(\pm 1, \pm 1, 0), (\pm 1, 0, \pm 1), (0, \pm 1, \pm 1)\}$

This stencil is widely used because it provides sufficient symmetry and velocity directions to accurately recover the Navier-Stokes equations in 3D, while being computationally more efficient than lattices with more velocity vectors (e.g., D3Q27).

5. Speculative 'Quantum-Inspired' Considerations for LBM D3Q19

The concept of a direct "Quantum LBM D3Q19" is not an established, practical scientific methodology in current fluid dynamics research. The Lattice Boltzmann Method is a classical mesoscopic simulation technique rooted in statistical mechanics, while quantum mechanics operates at fundamentally different scales and paradigms, describing phenomena at atomic and subatomic levels.

However, one can engage in speculative "quantum-inspired" approaches or explore how quantum computing might interact with LBM. These considerations are largely theoretical and represent nascent research avenues rather than concrete implementations for fluid simulation. The following points expand on the ideas presented in the provided Python code's `speculative_quantum_considerations` function.

5.1. Quantum Walks on the Lattice

Instead of classical particle distribution functions f_k , one could conceptually consider quantum states representing "particles" undergoing quantum walks on the LBM lattice. Each lattice site might host a quantum state (e.g., a qubit or qudit) whose internal degrees of freedom correspond to the velocity directions [21]. The "streaming" step could then be analogous to a quantum coin operation followed by a shift operator,

similar to discrete-time quantum walks [22]. The "collision" step would then be a local unitary operation on the quantum states at each node, potentially entangling them. This approach draws inspiration from the fundamental concepts of quantum information and quantum computation [12, 20].

5.2. Superposition of Distribution Functions

Imagine a scenario where the fluid system exists in a superposition of multiple classical LBM states (i.e., superpositions of different sets of f_k distributions across the lattice). A hypothetical quantum LBM might evolve these superpositions, allowing for the exploration of multiple fluid configurations simultaneously. This would necessitate representing f_k not as classical numbers but as amplitudes in a quantum state vector, evolving under unitary operators, a concept rooted in the principles of quantum mechanics [6, 10].

5.3. Entanglement and Non-Locality

Introducing entanglement between the quantum states at different lattice sites or between different velocity populations at the same site could be another speculative avenue. This might potentially capture non-local correlations in fluid dynamics that are difficult to model with classical LBM. However, the physical interpretation for fluid flow in such an entangled quantum LBM framework remains highly abstract and would require significant theoretical development, possibly drawing parallels with quantum field theory or quantum many-body physics [3, 8, 11].

5.4. Quantum Operators for Collision and Streaming

Replacing the classical BGK collision operator and streaming operations with their quantum mechanical counterparts (unitary operators) is a highly ambitious goal. The primary challenge lies in designing such operators that rigorously conserve relevant macroscopic quantities (mass, momentum, energy) in a quantum context and yield classical fluid dynamics in a "decoherence" limit. This would require a profound reformulation of the LBM from first quantum principles, potentially touching upon ideas from quantum field theory [5, 18].

5.5. Quantum Algorithms for LBM Sub-problems

A more practical and less speculative approach involves leveraging quantum computers to accelerate specific computationally intensive parts of a classical LBM. While not a "quantum LBM" itself, quantum algorithms could offer computational advantages. * **Solving Linear Systems:** More complex collision operators (e.g., Multiple Relaxation Time LBM) can involve solving linear systems, which might be accelerated by quantum algorithms like HHL [23]. * **Optimization:** Quantum annealing or variational quantum eigensolvers (VQE) could be used to optimize LBM parameters, boundary conditions, or even discover new lattice designs [17]. * **Quantum Machine Learning:** Quantum machine learning techniques could be applied to learn optimal LBM parameters, model complex fluid phenomena from data, or enhance turbulence closures [24]. * **Quantum Simulators:** Concepts from quantum simulators, which use controllable quantum systems to mimic other quantum systems, could inspire new classical models or computational paradigms [19].

This area of using quantum computing to enhance classical simulations is an active field of research [2, 4, 16].

5.6. Quantum Field Theory Analogies

Viewing the LBM populations as "fields" and exploring analogies with quantum field theory, where particles are excitations of underlying fields, is a highly theoretical perspective [3, 5, 11, 18]. This could potentially link LBM to fundamental physics, such as quantum gravity or quantum cosmology [9, 13, 15], but it is far removed from practical fluid simulation and primarily serves as a source of conceptual inspiration for novel computational models [14].

5.7. Conclusion on Quantum-Inspired LBM

The direct mapping of LBM to a quantum mechanical framework for fluid simulation is an open and extremely challenging research area. Most current "quantum LBM" discussions are either about using quantum computers to accelerate classical LBM computations, or they are highly theoretical explorations of quantum walks and quantum field theory on a lattice that might *inspire* new mesoscopic models, rather than directly

simulating fluid dynamics with quantum mechanics in a practical sense. The provided Python code is a standard classical D3Q19 LBM implementation, and the quantum considerations remain largely in the realm of theoretical physics and quantum computation research.

6. Conclusion

The Lattice Boltzmann Method, particularly the D3Q19 model, provides a robust and efficient mesoscopic approach for simulating complex fluid flows, including turbulent phenomena. Its ability to recover macroscopic Navier-Stokes equations from simple collision and streaming rules, combined with its computational advantages for parallelization and complex boundary conditions, makes it a valuable tool in computational fluid dynamics. High-fidelity simulations, such as DNS/512, are crucial for generating data to analyze characteristic turbulence statistics like isotropy, structure functions, energy spectra, and flatness, which reveal the intricate multi-scale and intermittent nature of turbulence.

While LBM is firmly rooted in classical statistical mechanics, the burgeoning field of quantum computing and quantum physics opens speculative avenues for "quantum-inspired" LBM. These range from conceptual re-formulations based on quantum walks and superpositions to more practical applications where quantum algorithms might accelerate specific components of classical LBM simulations. It is critical to distinguish between these highly theoretical inspirations and the direct quantum simulation of fluid dynamics, which remains a profound scientific challenge. Future research will likely see continued advancements in classical LBM for turbulence, alongside theoretical explorations and practical applications of quantum computing to enhance fluid dynamics simulations.

7. References

1. Bauer, M., & Rüde, U. (2018). An improved lattice Boltzmann D3Q19 method based on an alternative equilibrium discretization. *arXiv preprint arXiv:1803.04937v2*.

2. Kolmogorov, A. N. (1941). The local structure of turbulence in incompressible viscous fluid for very large Reynolds numbers. *Doklady Akademii Nauk SSSR*, 30(4), 301-305.
3. Foo, J., & Zych, M. (2023). Superpositions of thermalisations in relativistic quantum field theory. *arXiv preprint arXiv:2307.02593v2*. (Quantum Field Theory)
4. Goldberg, A. (2024). Quantum games and synchronicity. *arXiv preprint arXiv:2408.15444v4*. (Quantum Computing/Information)
5. Halimeh, J. C., Van Damme, M., Zache, T. V., Banerjee, D., & Hauke, P. (2021). Achieving the quantum field theory limit in far-from-equilibrium quantum link models. *arXiv preprint arXiv:2112.04501v3*. (Quantum Field Theory)
6. Martin-Dussaud, P. (2020). Searching for Coherent States: From Origins to Quantum Gravity. *arXiv preprint arXiv:2003.11810v4*. (Quantum Gravity/Foundations)
7. Ballesteros, A., Fernandez-Silvestre, D., Giacomini, F., & Gubitosi, G. (2025). Quantum Galilei group as quantum reference frame transformations. *arXiv preprint arXiv:2504.00569v3*. (Quantum Foundations)
8. Mackel, N. E., Yang, J., & del Campo, A. (2022). Quantum Alchemy and Universal Orthogonality Catastrophe in One-Dimensional Anyons. *arXiv preprint arXiv:2210.10776v3*. (Quantum Many-Body Physics)
9. Favalli, T., & Smerzi, A. (2022). A model of quantum spacetime. *arXiv preprint arXiv:2207.01005v3*. (Quantum Gravity)
10. D'Esposito, V., Fabiano, G., Frattulillo, D., & Mercati, F. (2024). Doubly Quantum Mechanics. *arXiv preprint arXiv:2412.05997v3*. (Quantum Foundations)
11. Chen, L. Q., Giacomini, F., & Rovelli, C. (2022). Quantum States of Fields for Quantum Split Sources. *arXiv preprint arXiv:2207.10592v2*. (Quantum Field Theory)
12. Barenco, A. (1995). A Universal Two--Bit Gate for Quantum Computation. *arXiv preprint quant-ph/9505016v1*. (Quantum Computing)
13. Kabel, V., de la Hamette, A. C., Castro-Ruiz, E., & Brukner, Č. (2022). Quantum conformal symmetries for spacetimes in superposition. *arXiv preprint arXiv:2207.00021v3*. (Quantum Gravity/Foundations)

14. Cirilo-Lombardo, D. J., & Sanchez, N. G. (2025). Classical (ontological) dual states in quantum theory and the minimal group representation Hilbert space. *arXiv preprint arXiv:2501.11119v1*. (Quantum Foundations)
15. Chataignier, L., Kiefer, C., & Moniz, P. (2023). Observations in Quantum Cosmology. *arXiv preprint arXiv:2306.14948v2*. (Quantum Cosmology)
16. Arrighi, P., Durbec, A., & Wilson, M. (2021). Quantum networks theory. *arXiv preprint arXiv:2110.10587v3*. (Quantum Information)
17. Sørensen, J. J. W. H., Pedersen, M. K., Munch, M., Haikka, P., Jensen, J. H., Planke, T., ... & Sherson, J. F. (2015). Exploring the Quantum Speed Limit with Computer Games. *arXiv preprint arXiv:1506.09091v3*. (Quantum Computing/Optimization)
18. Gisin, N., & Del Santo, F. (2023). Towards a measurement theory in QFT: "Impossible" quantum measurements are possible but not ideal. *arXiv preprint arXiv:2311.13644v2*. (Quantum Field Theory)
19. Vasilyev, D. V., Grankin, A., Baranov, M. A., Sieberer, L. M., & Zoller, P. (2020). Monitoring Quantum Simulators via Quantum Non-Demolition Couplings to Atomic Clock Qubits. *arXiv preprint arXiv:2006.00214v2*. (Quantum Computing/Simulators)
20. Deutsch, D., Barenco, A., & Ekert, A. (1995). Universality in Quantum Computation. *arXiv preprint quant-ph/9505018v1*. (Quantum Computing)
21. Tianyan Quantum Group. (2025). Tianyan: Cloud services with quantum advantage. *arXiv preprint arXiv:2512.10504v2*. (Quantum Computing)
22. Aharonov, Y., Davidovich, L., & Zagury, N. (1993). Quantum random walks. *Physical Review A*, 48(2), 1687.
23. Harrow, A. W., Hassidim, A., & Lloyd, S. (2009). Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15), 150502.
24. Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, 549(7671), 195-202.

References

7. References

1. Bauer, M., & R  de, U. (2018). An improved lattice Boltzmann D3Q19 method based on an alternative equilibrium discretization. *arXiv preprint arXiv:1803.04937v2*.
2. Kolmogorov, A. N. (1941). The local structure of turbulence in incompressible viscous fluid for very large Reynolds numbers. *Doklady Akademii Nauk SSSR*, 30(4), 301-305.
3. Foo, J., & Zych, M. (2023). Superpositions of thermalisations in relativistic quantum field theory. *arXiv preprint arXiv:2307.02593v2*. (Quantum Field Theory)
4. Goldberg, A. (2024). Quantum games and synchronicity. *arXiv preprint arXiv:2408.15444v4*. (Quantum Computing/Information)
5. Halimeh, J. C., Van Damme, M., Zache, T. V., Banerjee, D., & Hauke, P. (2021). Achieving the quantum field theory limit in far-from-equilibrium quantum link models. *arXiv preprint arXiv:2112.04501v3*. (Quantum Field Theory)
6. Martin-Dussaud, P. (2020). Searching for Coherent States: From Origins to Quantum Gravity. *arXiv preprint arXiv:2003.11810v4*. (Quantum Gravity/Foundations)
7. Ballesteros, A., Fernandez-Silvestre, D., Giacomini, F., & Gubitosi, G. (2025). Quantum Galilei group as quantum reference frame transformations. *arXiv preprint arXiv:2504.00569v3*. (Quantum Foundations)
8. Mackel, N. E., Yang, J., & del Campo, A. (2022). Quantum Alchemy and Universal Orthogonality Catastrophe in One-Dimensional Anyons. *arXiv preprint arXiv:2210.10776v3*. (Quantum Many-Body Physics)
9. Favalli, T., & Smerzi, A. (2022). A model of quantum spacetime. *arXiv preprint arXiv:2207.01005v3*. (Quantum Gravity)
10. D'Esposito, V., Fabiano, G., Frattulillo, D., & Mercati, F. (2024). Doubly Quantum Mechanics. *arXiv preprint arXiv:2412.05997v3*. (Quantum Foundations)

11. Chen, L. Q., Giacomini, F., & Rovelli, C. (2022). Quantum States of Fields for Quantum Split Sources. *arXiv preprint arXiv:2207.10592v2*. (Quantum Field Theory)
12. Barenco, A. (1995). A Universal Two--Bit Gate for Quantum Computation. *arXiv preprint quant-ph/9505016v1*. (Quantum Computing)
13. Kabel, V., de la Hamette, A. C., Castro-Ruiz, E., & Brukner, Č. (2022). Quantum conformal symmetries for spacetimes in superposition. *arXiv preprint arXiv:2207.00021v3*. (Quantum Gravity/Foundations)
14. Cirilo-Lombardo, D. J., & Sanchez, N. G. (2025). Classical (ontological) dual states in quantum theory and the minimal group representation Hilbert space. *arXiv preprint arXiv:2501.11119v1*. (Quantum Foundations)
15. Chataignier, L., Kiefer, C., & Moniz, P. (2023). Observations in Quantum Cosmology. *arXiv preprint arXiv:2306.14948v2*. (Quantum Cosmology)
16. Arrighi, P., Durbec, A., & Wilson, M. (2021). Quantum networks theory. *arXiv preprint arXiv:2110.10587v3*. (Quantum Information)
17. Sørensen, J. J. W. H., Pedersen, M. K., Munch, M., Haikka, P., Jensen, J. H., Planke, T., ... & Sherson, J. F. (2015). Exploring the Quantum Speed Limit with Computer Games. *arXiv preprint arXiv:1506.09091v3*. (Quantum Computing/Optimization)
18. Gisin, N., & Del Santo, F. (2023). Towards a measurement theory in QFT: "Impossible" quantum measurements are possible but not ideal. *arXiv preprint arXiv:2311.13644v2*. (Quantum Field Theory)
19. Vasilyev, D. V., Grankin, A., Baranov, M. A., Sieberer, L. M., & Zoller, P. (2020). Monitoring Quantum Simulators via Quantum Non-Demolition Couplings to Atomic Clock Qubits. *arXiv preprint arXiv:2006.00214v2*. (Quantum Computing/Simulators)
20. Deutsch, D., Barenco, A., & Ekert, A. (1995). Universality in Quantum Computation. *arXiv preprint quant-ph/9505018v1*. (Quantum Computing)
21. Tianyan Quantum Group. (2025). Tianyan: Cloud services with quantum advantage. *arXiv preprint arXiv:2512.10504v2*. (Quantum Computing)
22. Aharonov, Y., Davidovich, L., & Zagury, N. (1993). Quantum random walks. *Physical Review A*, 48(2), 1687.

23. Harrow, A. W., Hassidim, A., & Lloyd, S. (2009). Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15), 150502.
24. Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, 549(7671), 195-202.